🎓 DAY 1: MASTER PRESENTATION & SPEAKER NOTES

GLA University — High Performance Computing & Heterogeneous AI Systems

Course Code: GLA-HPC-BCSG701 | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Audience:** B.Tech / M.Tech Computer Science (60 Students)
Primary Reference: `Intel & NEC - High Performance Computing.pdf`

🏛️ 1. Syllabus Boundary & Official Scope Integrity

> [!IMPORTANT] **Strict Syllabus Boundary Rule:** Day 1 contains **strictly 7 topics** as defined in the official curriculum document. Topics like **OpenMP**, **MPI distributed execution**, **Machine Learning / oneDAL**, **TensorFlow-MKL**, **Slurm job queues**, **Horovod**, **Intel VTune Profiler**, and **LLM Fine-tuning** are explicitly scheduled for Days 2, 3, 4, and 5. Do NOT introduce Slurm sbatch scripts or multi-node MPI clusters on Day 1. Day 1 builds the impenetrable foundational bedrock: Linux mastery, Shell automation, SSH remote access, and oneAPI environment architecture.

```
+-------------------------------------------------------------------------------+
|                              DAY 1 SYLLABUS BOUNDARY                          |
+-------------------------------------------------------------------------------+
|  [DAY 1: IN-SCOPE]                                                            |
|  1. Introduction to HPC (Concept, Purpose, Clusters vs Workstations)          |
|  2. Intel HPC Architecture (Xeon Scalable CPUs, GPUs, Habana Gaudi AI Accelerators)     |
|  3. Linux Basics (Filesystem Hierarchy, Absolute/Relative Paths, Core CLI, Environment) |
|  4. Shell Scripting for Automation (Shebang, Variables, IO Redirection, Automation)    |
|  5. Secured Shell / SSH (Local vs Remote, Keys vs Passwords, HPC Gateway Access)       |
|  6. Intel oneAPI Toolkit (Unified Cross-Architecture Vision, DPCPP/SYCL, Libraries)    |
|  7. Introduction to Intel HPC Toolkit (Classic Compilers, ICX/DPCPP, HPC Environment)  |
+-------------------------------------------------------------------------------+
|  [DAY 2-5: FUTURE SCOPE - DO NOT PRE-EMPT ON DAY 1]                           |
|  OpenMP (Day 2) | MPI Cluster Comms (Day 2) | oneDAL ML (Day 2) | TensorFlow-MKL (Day 3)|
|  Slurm Batch Scheduler (Day 3/4) | Horovod Multi-GPU (Day 4) | VTune Profiler (Day 4)   |
|  Generative AI & Gaudi LLM Fine-Tuning (Day 5)                                |
+-------------------------------------------------------------------------------+
```

📋 2. Complete 7-Topic & Subtopic Breakdown

Topic 1: Introduction to HPC

PDF explicitly mentioned: "Introduction to HPC" - **HPC ka full form aur basic concept:** High Performance Computing — combining compute power to deliver much higher performance than a standard desktop or workstation. - **HPC systems ka purpose:** Solving grand-challenge problems requiring billions of floating-point operations per second (FLOPs) that cannot run on commodity PCs. - **HPC systems ka architectural**

overview: Interconnected compute nodes (head node + worker nodes), high-speed interconnects (InfiniBand / Omni-Path / RoCE), and shared parallel filesystems (Lustre / GPFS / NFS). - **Computationally intensive problems mein use:** Climate modeling, molecular dynamics, drug discovery, financial risk simulation, and large-scale AI training. - **HPC aur ordinary computing ka basic difference:** - *Ordinary Laptop:* Single socket, 8-16 GB RAM, 50-100 GB/s memory bandwidth, local SSD bottleneck, consumer OS scheduler. - *HPC Node:* Multi-socket (e.g. Dual Intel Xeon Gold 6442Y), 256-512 GB DDR5 ECC RAM, 300+ GB/s memory bandwidth, NUMA domains, interconnects delivering sub-microsecond latency. - **HPC environment ka introductory overview:** Head node (login node) where users write scripts, compute nodes where batch computations execute.

Topic 2: Intel HPC Architecture

PDF explicitly mentioned: "Xeon, GPUs, AI accelerators" - **Intel HPC architecture ka overview:** Heterogeneous computing architecture where different silicon engines handle the workloads they are physically best suited for. - **Intel Xeon processors ka introduction:** 4th & 5th Gen Intel Xeon Scalable Processors (Sapphire Rapids / Emerald Rapids) featuring 32 to 64 physical cores, AVX-512 vector units, and Intel AMX (Advanced Matrix Extensions) for on-die matrix multiply acceleration. - **CPU ka role in HPC:** Master orchestrator, high-clock sequential execution, data preprocessing, graph traversals, and high-precision scientific math (FP64). - **GPU ka introduction aur HPC mein role:** Intel Data Center GPU Max Series (Ponte Vecchio) featuring thousands of vector/matrix compute engines (Xe-Cores) executing millions of concurrent threads for FP32/FP16 matrix multiplication and Deep Learning. - **AI accelerators ka introduction:** Habana Gaudi 2 / Gaudi 3 processors engineered from silicon up for tensor contraction, bfloat16 GEMM, and on-die integrated 24x 100GbE RoCE RDMA networking. - **CPU, GPU aur AI accelerator ka basic distinction:** | Dimension | Intel Xeon CPU | Intel Data Center GPU | Habana Gaudi AI Accelerator | | :--- | :--- | :--- | :--- | | **Core Architecture** | Few (32-64), heavy, high clock cores | Thousands of lightweight SIMD vector cores | Specialized Tensor Processor Cores (TPCs) + Matrix Engines (MME) | | **Latency vs Throughput** | Optimized for minimum latency | Optimized for maximum throughput | Optimized for tensor GEMM & memory bandwidth | | **Ideal Workload** | OS, control flow, preprocessing, scalar/vector math | Massive matrix operations, rendering, parallel simulation | Large Language Models (LLMs), Transformer training & inference | - **HPC hardware components ka introductory overview:** Host CPU, PCIe Gen5 buses, CXL memory expansion, HBM (High Bandwidth Memory), and RDMA network interface cards (NICs).

Topic 3: Linux Basics

PDF explicitly mentioned: "File system, commands, environment setup"

A. Linux Filesystem

- **Files aur directories ka concept:** In UNIX/Linux, *Everything is a file* (files, directories, devices, sockets).
- **Absolute vs Relative paths:**
- *Absolute:* Starts from the root `/` (e.g. `/home/user/project/script.sh`). Unambiguous from any directory.
- *Relative:* Relative to Current Working Directory (CWD) (e.g. `./script.sh`, `../data/input.csv`).
- **Directory structure ko navigate karna:** Understanding the FHS (Filesystem Hierarchy Standard):
- `/` (Root directory)
- `/home/<username>` (User's private workspace; quota-limited)
- `/scratch` or `/tmp` (High-speed ephemeral storage for running jobs)
- `/opt` (Installed third-party packages, e.g. `/opt/intel/oneapi`)

- `/bin`, `/usr/bin` (Essential binaries and executables)
- `/etc` (System-wide configuration files)
- **Files aur folders ki location samajhna:** Using `pwd` (print working directory) to continuously verify location.

B. Linux Commands

- **Current directory & navigation:** `pwd`, `cd`, `cd ..`, `cd ~`, `cd -`.
- **Listing contents:** `ls`, `ls -l` (detailed permissions/sizes), `ls -la` (including hidden `.` files), `ls -lh` (human-readable sizes).
- **Creating files & directories:** `mkdir -p project/src/data`, `touch main.cpp`, `cat << 'EOF' > file.txt`.
- **Copying, moving & removing:**
- `cp file.txt copy.txt`, `cp -r src_dir/ dest_dir/`
- `mv old_name.txt new_name.txt` (renaming/moving)
- `rm file.txt`, `rm -r directory/` (Safety warning: `rm` has no recycle bin in Linux!).
- **Viewing file contents:** `cat`, `head -n 20`, `tail -n 20`, `less`, `grep -i "keyword" file.log`.
- **Basic command syntax:** `command -options arguments` (e.g., `tar -czvf archive.tar.gz folder/`).

C. Environment Setup

- **Linux working environment:** Shell session, standard input (stdin 0), standard output (stdout 1), standard error (stderr 2).
- **User environment:** Current shell (`echo $SHELL`), current user (`whoami`), host identity (`hostname`).
- **Environment variables ka introductory concept:**
- What is an environment variable? A dynamic named value stored in system memory affecting process execution.
- Viewing variables: `env`, `printenv`, `echo $PATH`, `echo $LD_LIBRARY_PATH`.
- Setting temporary variables: `export MY_VAR="HPC_2026"`.
- The `$PATH` variable: How Linux searches for executable commands without specifying `/usr/bin/python3`.
- **Working environment ko prepare karna:** Editing configuration files like `~/.bashrc` to persist environment settings across login sessions.

Topic 4: Shell Scripting for Automation

PDF explicitly mentioned: "Shell scripting for automation" - **Shell aur shell scripting ka introduction:** Why script? Reproducibility, batch execution, eliminating human typing error in 50-step workflows. - **Shell script file banana:** Creating `automate.sh` with a text editor (`nano`, `vim`, or bash redirection). - **The Shebang (`#!/bin/bash`):** Directing the kernel's `execve` system call to parse the script with the Bourne-Again SHell. - **Basic script structure:** `bash #!/bin/bash # Exit immediately if a command exits with a non-zero status set -e echo "[INFO] Starting automated workflow at $(date)..."` - **Variables ka basic use:** - Defining: `DATASET_DIR="/scratch/dataset"` (No spaces around `=`). - Accessing: `echo "Processing data in ${DATASET_DIR}"`. - **Input aur output ka basic concept:** - Output redirection: `command > output.log` (overwrite), `command >> output.log` (append). - Error redirection: `command 2> error.log`, combined: `command > all.log 2>&1`. - Positional arguments: `$1`, `$2`, `$#` (count), `$@` (all arguments). - **Sequential command execution:** `;` (unconditional), `&&` (execute next only if previous succeeded), `||` (execute if previous failed). - **Script permissions:** Understanding `chmod +x script.sh` (`rwx` octal: 755 = read/write/execute for owner, read/execute for others). - **Simple task automation:** Automating directory creation, compiling a code file, and verifying its return code (`$?`).

Topic 5: Secured Shell (SSH)

PDF explicitly mentioned: "Secured Shell aur secure remote access" - **SSH ka full form aur purpose:** Secure Shell (Network protocol running on TCP Port 22) providing encrypted communication over an insecure network. - **Local vs Remote system:** The student's local laptop/workstation vs the remote HPC Head Node sitting in the server rack. - **SSH Client aur SSH Server:** OpenSSH client (`ssh`) running on user's machine connecting to OpenSSH daemon (`sshd`) running on the remote cluster. - **Remote system se secure connection:** Syntax: `bash ssh username@remote_host_or_ip -p 22` - **Authentication mechanisms:** - *Password-based:* Subject to brute-force attacks, requires interactive human input (cannot easily automate in CI/CD). - *Key-based (Public/Private Key Pair):* Cryptographic proof using asymmetric encryption (`ed25519` or `rsa-4096`). - **Key-based authentication workflow:** 1. Generate key pair on local machine: `ssh-keygen -t ed25519 -C "shivam@gla"` 2. Public key (`id_ed25519.pub`) is placed into remote node's `~/.ssh/authorized_keys` . 3. Private key (`id_ed25519`) NEVER leaves the local machine (permissions strictly `600`). 4. Server sends a cryptographic challenge encrypted with public key; client decrypts with private key. - **Closing remote sessions:** `exit` or `Ctrl + D` . - **HPC environment mein SSH ka role:** Head node is exclusively accessed via SSH; compute nodes communicate internally using passwordless SSH keys for distributed execution.

Topic 6: Intel oneAPI Toolkit

PDF explicitly mentioned: "Intel oneAPI toolkit" - **Intel oneAPI toolkit ka introduction:** An open, standards-based, cross-architecture programming model delivering unified developer productivity across scalar (CPU), vector (GPU), matrix (AI), and spatial (FPGA) architectures. - **oneAPI ka purpose aur HPC se relation:** Eliminates proprietary hardware lock-in (e.g. proprietary CUDA code that only runs on one vendor's GPUs). Developers write high-performance C++ code using **SYCL** (Standard C++ heterogeneous abstraction) that compiles across Intel, NVIDIA, and AMD hardware. - **Toolkit aur development environment ka concept:** A curated suite of optimized compilers, domain-specific libraries (oneMKL for math, oneDNN for deep learning, oneDAL for data analytics), and analysis tools. - **Available toolkit components:** - *Intel DPC++/C++ Compiler (`icx` / `icpx`):* LLVM-based C++ compiler with SYCL and OpenMP support. - *Intel oneMKL (Math Kernel Library):* Heavily vector-optimized BLAS, LAPACK, FFT routines tuned for AVX-512 and AMX. - *Intel oneDAL (Data Analytics Library):* Accelerated algorithms for classical Machine Learning (PCA, SVM, k-Means). - *Intel oneDNN (Deep Neural Network Library):* Silicon-tuned primitives (convolutions, activations, normalizations). - **oneAPI environment ka setup:** Activating the environment using the environment script: `bash source /opt/intel/oneapi/setvars.sh` - **Installed tools ko access karna:** Verifying compilers via `icpx --version` , `sycl-ls` (listing available SYCL hardware compute devices).

Topic 7: Introduction to Intel HPC Toolkit

PDF explicitly mentioned: "Introduction to Intel HPC Toolkit" - **Intel HPC Toolkit ka introduction:** The specialized add-on toolkit designed specifically for building, analyzing, and scaling compute-intensive parallel applications across distributed clusters. - **HPC development aur execution workflow:** `Code Written (C++/SYCL/Fortran) ⟶ Compile with Intel Compiler (icx/ifx) ⟶ Profile / Debug ⟶ Cluster Execution` - **Toolkit ka HPC environment mein role:** - *Intel Fortran Compiler (`ifx`):* Modern Fortran compiler for legacy scientific simulations. - *Intel MPI Library (`mpiicx` / `mpirun`):* Multi-fabric message passing library optimized for low-latency interconnects. - *Intel Inspector:* Memory and thread error debugger to detect race conditions and memory leaks in parallel code. - *Intel Trace Analyzer and Collector (ITAC):* Visualizing MPI communication bottlenecks across cluster nodes. - **Toolkit ko available HPC environment se relate karna:** How the HPC Toolkit sits on top of the Base oneAPI Toolkit, transforming a single workstation compiler into a distributed supercomputing stack.

✅ 3. Day 1 Study & Coverage Checklist (0/22 Interactive Tracker)

```
### 1. Introduction to HPC
- [ ] 01. HPC concept and purpose
- [ ] 02. HPC system overview
- [ ] 03. HPC versus ordinary computing

### 2. Intel HPC Architecture
- [ ] 04. Intel Xeon
- [ ] 05. GPUs
- [ ] 06. AI accelerators
- [ ] 07. Hardware roles

### 3. Linux Basics
- [ ] 08. Filesystem and paths
- [ ] 09. Basic commands
- [ ] 10. Environment setup

### 4. Shell Scripting
- [ ] 11. Script creation and execution
- [ ] 12. Variables and commands
- [ ] 13. Task automation

### 5. SSH
- [ ] 14. Remote access
- [ ] 15. Connection syntax
- [ ] 16. Authentication basics

### 6. Intel oneAPI Toolkit
- [ ] 17. Toolkit purpose
- [ ] 18. Components overview
- [ ] 19. Environment access

### 7. Intel HPC Toolkit
- [ ] 20. Toolkit introduction
- [ ] 21. HPC workflow
- [ ] 22. Available tools
```

💻 4. Slide-by-Slide Speaker Notes & Presentation Guide

Mapped to the 15 Slides of `Day_01_NotebookLM_Master_Deck.pptx`

Slide 1: Title & Welcome

- **Slide Title:** High Performance Computing: A Practical Approach
- **Visual:** Sleek dark-mode aesthetic with Intel & NEC Supercomputing Alliance branding.
- **Trainer Script (Hinglish/English):**

> "Good morning everyone, welcome to the intensive GLA University High Performance Computing and Heterogeneous AI Systems training! Over the next 5 days, we are going to

> bridge the gap between running models on your personal laptops and engineering high-throughput, cluster-scale applications. Today is Day 1: we build the foundation that every supercomputer engineer in the world relies on."

- **Key Takeaway:** Establishing professional engineering standards from minute 1.

Slide 2: AI Engineering: Workstation to HPC Cluster

- **Slide Title:** From Workstation to HPC Cluster
- **Visual:** Side-by-side comparison diagram: Single-Node Laptop vs Distributed Cluster.
- **Trainer Script:**

> "Look at your laptop screen right now. If you train a small model on 10,000 rows, it works fine. But what happens when you train on 500 million parameters or 50 Terabytes of scientific data? Your laptop runs out of RAM, your CPU thermal-throttles, and the job crashes. In HPC, we don't buy a 100x bigger laptop — we network dozens or hundreds of enterprise compute nodes together with high-speed interconnects. Today, you learn how to control this hardware."

- **Checklist Covered:** #01, #02, #03.

Slide 3: Our Pedagogical Engine

- **Slide Title:** Pedagogical Engine: Concept -> System -> Hands-On
- **Visual:** Triangular execution engine: [01] Concept -> [02] Silicon Understanding -> [03] Bare-Metal Hands-On.
- **Trainer Script:**

> "In this workshop, we follow a 3-step discipline for every single topic: First, understand the mathematical and computational concept. Second, understand how the physical silicon (the CPU registers, cache lines, and vector units) executes it. Third, open the terminal and run real code. No hand-waving, no placeholders."

Slide 4: Core Pillars of Training

- **Slide Title:** The Core Pillars of Training
- **Visual:** Four pillar cards: Industry-Grade Tools, Performance Focus, Cluster-Based Learning, Project-Oriented.
- **Trainer Script:**

> "We are not using toy environments. You will use the exact toolchain used on the world's fastest supercomputers: Intel oneAPI, icpx, DPC++, and Linux command-line tooling."

Slide 5: The Software & Hardware Ecosystem

- **Slide Title:** Compute, Orchestration, and Abstraction
- **Visual:** Layered stack: Compute Silicon (Xeon, Max GPU, Gaudi) -> Linux & Network -> oneAPI Abstraction -> Applications.

- **Trainer Script:**

> "Notice how oneAPI acts as the universal bridge. Whether you target an Intel Xeon CPU, an Intel Data Center GPU, or an AI accelerator, oneAPI gives you a unified programming model."

Slide 6: The 5-Day Curriculum Arc

- **Slide Title:** The 5-Day Arc
- **Visual:** 5 sequential milestone nodes: Day 1 (You Are Here) to Day 5 (Generative AI on Gaudi).
- **Trainer Script:**

> "Notice our strict progression. Today is Day 1: Fundamentals of HPC, Linux, Shell & Toolkits. Tomorrow on Day 2, we dive into OpenMP and MPI. Day 3 is Deep Learning and Slurm. Day 4 is Distributed AI and VTune. And on Day 5, we do Generative AI with Habana Gaudi and OpenVINO. Today, master the foundation!"

Slide 7: Day 1 Agenda & Module Initialization

- **Slide Title:** `> init --module day_01_fundamentals`
- **Visual:** Terminal-style status screen showing 5 loaded modules.
- **Trainer Script:**

> "Here is our exact roadmap for today: We deconstruct HPC architecture, examine the hardware continuum, master Linux filesystem navigation, write shell scripts for automation, connect securely via SSH, and initialize the Intel oneAPI development environment."

Slide 8: Deconstructing HPC: Beyond the Laptop

- **Slide Title:** Deconstructing HPC: Beyond the Laptop
- **Visual:** Detailed hardware comparison: Memory bus width, NUMA domains, interconnects.
- **Trainer Script:**

> "Why can't you just add more RAM to your laptop? Because of the Memory Wall. A consumer Intel Core i7 processor has 2 memory channels giving roughly 80 GB/s bandwidth. A Dual-Socket Intel Xeon Scalable processor has 16 DDR5 memory channels giving over 600 GB/s bandwidth! In HPC, data bandwidth is just as important as clock speed."

- **Checklist Covered:** #01, #02, #03.

Slide 9: The Hardware Continuum: Xeon, GPUs & Gaudi

- **Slide Title:** The Hardware Continuum
- **Visual:** Three-column table: Intel Xeon CPUs, Intel GPUs (Max Series), AI Accelerators (Habana Gaudi).
- **Trainer Script:**

> "Let's distinguish these three silicon engines clearly: - **Intel Xeon CPUs:** The master orchestrators. High single-thread clock speed, large L3 caches, AVX-512 and AMX for mixed-precision matrix math. - **Intel GPUs:** Built for massive parallelism. When you have thousands of matrix tiles that can be calculated independently, the GPU dominates. - **Habana Gaudi Accelerators:** Purpose-built for AI. They strip away graphics rasterizers and fill the die with Tensor Processor Cores and on-die 100Gb Ethernet RDMA."

- **Checklist Covered:** #04, #05, #06, #07.

Slide 10: Environment Navigation: Linux & The Filesystem

- **Slide Title:** Linux & The Filesystem: Hot vs Cold Storage
- **Visual:** FHS diagram showing `/home`, `/scratch`, and command syntax.
- **Trainer Script:**

> "In an HPC cluster, where you save your files matters! - `/home` is your 'Cold Zone' — standard quota, networked NFS, great for small scripts and code. - `/scratch` is your 'Hot Zone' — high-speed parallel NVMe storage. When training models or writing simulation output, you MUST point your scripts to scratch, or you will saturate the login node's network!"

- **Checklist Covered:** #08, #09, #10.

Slide 11: Access & Automation: SSH & Shell Scripting

- **Slide Title:** Access & Automation: SSH & Shell Scripts
- **Visual:** Client laptop -> SSH encrypted tunnel (Port 22) -> Head Node -> Worker Nodes.
- **Trainer Script:**

> "No one walks into an HPC datacenter with a monitor and keyboard. Everything happens over SSH. And no one types commands manually 50 times — we write shell scripts with shebang lines, error traps (`set -e`), and automated logging. Let's see how this works in real life."

- **Checklist Covered:** #11, #12, #13, #14, #15, #16.

Slide 12: The Software Bridge: Intel oneAPI

- **Slide Title:** The Software Bridge: Intel oneAPI
- **Visual:** Cross-architecture abstraction layer diagram: One programming model targeting CPU, GPU, and FPGA.
- **Trainer Script:**

> "Before oneAPI, if you wrote code for an NVIDIA GPU, you had to use proprietary CUDA. If you ported to an Intel GPU or CPU, you had to rewrite everything. Intel oneAPI adopts the open Khronos SYCL standard. You write clean modern C++ with SYCL, and it compiles seamlessly across all target silicon."

- **Checklist Covered:** #17, #18, #19.

Slide 13: Inside the Intel HPC Toolkit

- **Slide Title:** Inside the Intel HPC Toolkit
- **Visual:** Layered architecture showing Base Toolkit + HPC Toolkit components (`icx` , `ifx` , `Intel MPI` , `Inspector`).
- **Trainer Script:**

> "The Intel HPC Toolkit builds directly on top of the Base Toolkit. It provides the heavy industrial tooling: the Intel C++ and Fortran Compilers (`icx` , `ifx`), the Intel MPI library for cluster communication, and Intel Inspector for thread debugging."

- **Checklist Covered:** #20, #21, #22.

Slide 14: Day 1 Synthesis: Architecture of Execution

- **Slide Title:** Day 1 Synthesis: The Architecture of Execution
- **Visual:** Complete 5-step pipeline: SSH Login -> Linux Navigation -> Shell Automation -> oneAPI Compilation -> Hardware Execution.
- **Trainer Script:**

> "Look at the complete loop we have mastered today: We connected over SSH, navigated the Linux filesystem, wrote a shell script to automate execution, compiled code with Intel oneAPI, and executed on enterprise Intel silicon!"

Slide 15: Foundation Established & Transition to Day 2

- **Slide Title:** Foundation Established -> Loading Day 2
- **Visual:** Preview of Day 2: OpenMP shared-memory multithreading, MPI message passing, and Intel oneDAL machine learning acceleration.
- **Trainer Script:**

> "Congratulations! Your foundation is 100% solid. Tomorrow on Day 2, we unleash the true power of multiple CPU cores using OpenMP and distribute across multiple nodes with MPI. See you tomorrow morning!"

🔬 5. Live Demonstration & Lab Synchronization Playbook

Minute	Activity	Delivery Track	Exact Command / Cell to Execute
00 - 20	Course Kickoff & HPC Concept	Classroom Slides	Slides 1 - 6 (Pedagogy, Scope, 5-Day Arc)
20 - 45	HPC Architecture (Xeon, GPU, Gaudi)	Colab + Projector	**Colab Cell 1 & 2:** Run `lscpu` , inspect AVX-512 & AMX flags

Minute	Activity	Delivery Track	Exact Command / Cell to Execute
45 - 75	Linux Basics (Paths, Commands, Env)	Colab Hands-On	**Colab Cell 3 - 6:** Directory creation, absolute vs relative paths, `env`, `$PATH`
75 - 105	Shell Scripting Automation	Colab Hands-On	**Colab Cell 7 & 8:** Run `hpc_node_audit.sh` automated node inspection script
105 - 120	*15-Minute Rejuvenation Break*	Break	Students review notes, check Colab connectivity
120 - 145	SSH Remote Access & Cluster Gateway	Live IDC Cluster Demo	**Trainer Terminal:** `ssh idc-vm`, show real Intel Xeon Gold 6442Y node
145 - 170	Intel oneAPI & HPC Toolkit Setup	Colab + Terminal	**Colab Cell 10 - 14:** `source setvars.sh`, compile SYCL vector add
170 - 180	Daily MCQ Challenge & Q&A	Colab + Classroom	**Colab Cell 25 & 26:** 5 Interactive MCQs + Review for Day 2

🎯 6. Common Student Pitfalls & Proactive Trainer Fixes

1. **Pitfall:** Student types `cd project` instead of `cd ~/project` and gets `No such file or directory`.
2. **Trainer Explanation:** Explain Relative vs Absolute paths. Show that `pwd` must be checked before using relative paths.
3. **Pitfall:** Student creates a shell script and gets `bash: ./script.sh: Permission denied`.
4. **Trainer Explanation:** Explain file mode bits. Run `ls -l script.sh` to show missing `x` flag, then `chmod +x script.sh`.
5. **Pitfall:** Student sets `VAR = "value"` (with spaces around `=`) and gets `command not found: VAR`.
6. **Trainer Explanation:** Bash treats spaces as argument delimiters. Show syntax: `VAR="value"`.
7. **Pitfall:** Student asks: "*Can we run Slurm jobs today?*"
8. **Trainer Explanation:** Reinforce the syllabus boundary: "*Slurm is our enterprise batch queue scheduler which we will master on Day 3 and 4 once our parallel code is written with OpenMP and MPI!*"